Proyecto: TuRecorrido

Integrantes: Vicente Herrera y Nicolás Irarrázabal

Paso 1: Análisis del contexto:

TuRecorrido será una App la cual dará apoyo a las personas que suelen usar el transporte público,

específicamente desde Puerto Montt y Puerto Varas. Donde un pasajero podrá tener acceso a las distintas rutas del recorrido el cual eligió, no solo eso sino que ahora pueden reportar incidencias de forma anónima sobre acontecimientos que pasaron en su recorrido. Junto con la capacidad de poder obtener información acerca de la velocidad del microbús, descuentos en pasajes y mucho.

Actualmente contamos con catorce tablas las cuales completan una relación paralela para poder obtener información y guardarla adecuadamente.

Alcance:

- **Telemetría:** Tener un registro sobre el recorrido completo que hace el microbús, obtener su kilometraje por hora.
- **Incidencias:** Tener una forma de poder guardar y mostrar "Tickets" sobre problemas los cuales pasaron los pasajeros.
- **Tarifas:** Calcular de acuerdo a los porcentajes de descuento para cada tipo de pasajeros.
- **Horarios:** Obtener día actual para para apoyar horarios ya registrados.

Actualmente las tablas ya se poseen restricciones como: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL y CHECK. Estas controlan referencias, duplicados y valores permitidos. Los triggers de este avance complementan esas restricciones con reglas sobre los tickets.

Reglas de negocio utilizadas

- Un ticket debe relacionarse con un usuario y un recorrido existentes, esto se controla con claves foráneas.

- Todo ticket nuevo comienza en PENDIENTE y recibe su fecha de creación (fecha del sistema).

- Los cambios permitidos son: PENDIENTE a EN_PROCESO o DESESTIMADO, y EN_PROCESO a RESUELTO o DESESTIMADO. Si el estado queda igual, se permite.

- Cada ruta tiene como máximo una tarifa base registrada, mayor que cero. El descuento del tipo de pasajero está entre 0 y 100.

- Un conductor o vehículo se informa con reincidencia cuando su cantidad de tickets en el período supera el límite recibido. Se cuentan todos los estados, sirve para priorizar una revisión, no para acreditar una falta.

Paso 2: Diseño de procedimientos y funciones:

Se proponen tres funciones y dos procedimientos, con y sin parámetros. Los siguientes bloques son pseudocódigo de diseño:

- fn_dia_semana_actual — función sin parámetros
- fn_velocidad_promedio — función con parámetro
- fn_calcular_tarifa — función con parámetros
- sp_evaluar_reincidencia — procedimiento con parámetros
- sp_revisar_reincidencia_diaria — procedimiento sin parámetros

1. fn_dia_semana_actual — función sin parámetros

Propósito: devolver el día actual, de LUNES a DOMINGO, para consultar los horarios del día. No recibe parámetros, devuelve VARCHAR2 y usa la fecha del servidor con SYSDATE.

1 Calcular el índice: v_indice := TRUNC(SYSDATE) - TRUNC(SYSDATE, 'IW') + 1;
2. Convertir el índice con un CASE: 1: 'LUNES', 2: 'MARTES' etc.
3. Devolver el nombre con RETURN.

Así los lunes siempre valen 1, sin depender de la configuración del equipo. Es el mismo criterio de días que usamos con el VARRAY de la evaluación anterior.

Ejemplo:
SELECT id_ruta, hora_salida FROM horario WHERE dia_semana = fn_dia_semana_actual();

2. fn_velocidad_promedio — función con parámetro
Propósito: obtener el promedio de velocidades registradas para un recorrido. Entra p_id_recorrido IN VARCHAR2 y devuelve NUMBER con dos decimales.

1. SELECT ROUND(AVG(velocidad_kmh), 2) INTO v_promedio FROM telemetria_vehiculo WHERE id_recorrido = p_id_recorrido;

2. RETURN v_promedio;

AVG ignora los NULL: si el recorrido no tiene velocidades, devuelve NULL, que significa "sin datos" y no velocidad 0. Un id que no exista también devuelve NULL porque no tiene registros. La consulta sin GROUP BY entrega una fila, así que no hay NO_DATA_FOUND. La función se ubica en pkg_telemetria.

3. fn_calcular_tarifa — función con parámetros

Propósito: calcular cuánto paga un usuario en una ruta según su tipo de pasajero. Entran p_id_ruta IN VARCHAR2 y p_id_usuario IN VARCHAR2; devuelve NUMBER en pesos chilenos, redondeado al entero.

Cada búsqueda se hace en su propio bloque con NO_DATA_FOUND, para distinguir qué falló. La tabla tarifa ya garantiza máximo una tarifa por ruta y el CHECK deja el descuento entre 0 y 100. La función se ubica en pkg_tarifas.

4. sp_evaluar_reincidencia — procedimiento con parámetros
Propósito: contar los tickets de un conductor o vehículo en un período y devolver si supera el límite.

- p_tipo_objetivo IN VARCHAR2: 'CONDUCTOR' o 'VEHICULO'
- p_id_objetivo IN VARCHAR2: identificador del conductor o vehículo
- p_fecha_inicio IN TIMESTAMP y p_fecha_fin IN TIMESTAMP: límites del período
- p_limite IN NUMBER: entero mayor o igual que cero
- p_cantidad OUT NUMBER: cantidad de tickets
- p_supera_limite OUT VARCHAR2: 'SI' o 'NO'

El período incluye el instante inicial y excluye el final. Para un día completo se usa la medianoche de ese día como inicio y la del día siguiente como fin.

Si no tiene tickets devuelve 0 y 'NO'. Los errores se comunican con RAISE_APPLICATION_ERROR y mensajes específicos. Se usa un procedimiento porque los OUT entregan claramente los dos resultados.

5. sp_revisar_reincidencia_diaria — procedimiento sin parámetros
Propósito: mostrar cuántos tickets lleva hoy cada conductor. No recibe parámetros y no guarda reporte en ninguna tabla, solo muestra con DBMS_OUTPUT. Usa un límite de 10 tickets diarios, que es un valor de demostración y debe acordarse con el negocio.

Se ejecuta a mano, no hay planificador de tareas. Para ver los mensajes hay que habilitar SERVEROUTPUT en el cliente.

Organización y manejo de errores
- **pkg_telemetria**: contiene fn_velocidad_promedio (especificación y cuerpo).
- **pkg_tarifas**: contiene fn_calcular_tarifa (especificación y cuerpo).
- fn_dia_semana_actual, sp_evaluar_reincidencia y sp_revisar_reincidencia_diaria: objetos sueltos, para mantenerlo simple.
- **NO_DATA_FOUND** se usa cuando un SELECT INTO no encuentra filas. COUNT(*) devuelve 0 y AVG devuelve NULL, esos son resultados, no errores.
- Si usamos WHEN OTHERS, mostramos SQLCODE y SQLERRM y luego RAISE, para no ocultar el error.

Paso 3: Diseño de Triggers:
Introducción: Se proponen tres triggers sobre ticket_incidencia: dos de fila para las reglas del ticket y uno de sentencia como apoyo de diagnóstico en las pruebas.

1. trg_ticket_estado_inicial — nivel de fila
- Tabla: ticket_incidencia. Evento: BEFORE INSERT. Nivel: FOR EACH ROW.

- Propósito: a cada ticket nuevo ponerle estado PENDIENTE y la fecha actual.

```
:NEW.estado_revision := 'PENDIENTE';
:NEW.fecha_creacion  := SYSTIMESTAMP;
```

Se usa BEFORE porque modifica los valores antes de guardarlos, y FOR EACH ROW porque es por cada ticket.

2. trg_ticket_valida_cambio — nivel de fila

- Tabla: ticket_incidencia. Evento: BEFORE UPDATE OF estado_revision. Nivel: FOR EACH ROW.
- Propósito: dejar pasar solo las transiciones permitidas, sin bloquear actualizaciones que dejan el estado igual.

```
si :NEW.estado_revision es NULL                    -> rechazar
si :NEW.estado_revision = :OLD.estado_revision          -> permitir
PENDIENTE  -> EN_PROCESO o DESESTIMADO             -> permitir
EN_PROCESO -> RESUELTO o DESESTIMADO               -> permitir
```

Si el UPDATE no toca estado_revision, el trigger no se dispara. RESUELTO y DESESTIMADO no cambian a otro estado. Se usa BEFORE para rechazar antes de aplicar.

3. trg_ticket_aviso_insercion — nivel de sentencia

- Tabla: ticket_incidencia. Evento: AFTER INSERT. Nivel: sentencia, sin FOR EACH ROW.
- Propósito: mostrar un aviso cuando se ejecuta un INSERT sobre tickets, para ver la diferencia entre un trigger de fila y uno de sentencia.

```
DBMS_OUTPUT.PUT_LINE('Se ejecutó una sentencia INSERT sobre tickets.');
```

Se ejecuta una vez por sentencia, igual si inserta una fila que varias. No usa :OLD ni :NEW ni SQL%ROWCOUNT. Solo sirve de prueba.

Paso 4: Justificación técnica:

Problema y contexto

- Información de rutas y horarios repartida en Puerto Montt y Puerto Varas.
- No había un canal formal para registrar reclamos y darles seguimiento.
- TuRecorrido da una solución a los problemas de los pasajeros

Alcance del avance

- Ya existe un modelo de 14 tablas en Oracle.
- Qué hace este avance: velocidad promedio, tarifa por usuario, conteo de tickets y día actual.
- Tres funciones, dos procedimientos y tres triggers simples.
- Usa las mismas estructuras que veníamos trabajando.

Funciones

- fn_dia_semana_actual: no recibe parámetros, depende de la fecha del servidor.
- fn_velocidad_promedio: resume muchos parámetros de velocidad, NULL significa que no hay velocidades.
- fn_calcular_tarifa: aplica el descuento del tipo de pasajero y devuelve un monto.
- Ventaja: sus valores se pueden usar dentro de un SELECT.
- Limitación: si se llaman por cada fila de un reporte grande, repiten la consulta y tardan más.

Procedimientos

- sp_evaluar_reincidencia: recibe objetivo, período y límite, devuelve cantidad e indicador con OUT.
- sp_revisar_reincidencia_diaria: sin parámetros, recorre conductores con un cursor.
- Ventaja: separan el proceso y evitan repetir la lógica de conteo.
- Limitación: no se usan dentro de un SELECT, el cursor consulta por cada conductor.

Triggers de fila

- BEFORE INSERT: asigna el estado inicial y la fecha de cada ticket.
- BEFORE UPDATE OF estado_revision: valida las transiciones con :OLD y :NEW.
- Ventaja: la regla se aplica aunque el cambio venga de otro programa.
- Limitación: se ejecutan automáticamente y hay que considerarlos al investigar errores.
- Actualizar sin cambiar el estado se permite.

Trigger de sentencia

- AFTER INSERT sin FOR EACH ROW: un aviso por sentencia con DBMS_OUTPUT.
- Ventaja: un solo mensaje aunque se inserten varias filas.
- Limitaciones: no cuenta filas, no guarda información y requiere habilitar la salida en el cliente.
- Es apoyo de pruebas

Estructuras de la evaluación anterior

- RECORD: agrupa los datos de un resumen de recorrido, hay que definirlo antes de usarlo.
- VARRAY: representa los siete días, tiene un máximo de elementos definido.
- Cursores: permiten recorrer conductores y recorridos, los ciclos anidados aumentan el trabajo fila a fila.
- AVG y COUNT resuelven los cálculos sin recorrer manualmente cada medición o ticket.

Integridad y excepciones

- PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK y NOT NULL controlan la integridad básica.
- Los triggers complementan esas restricciones con la inicialización y las transiciones del ticket.
- NO_DATA_FOUND distingue búsquedas sin datos; RAISE_APPLICATION_ERROR comunica errores de negocio.
- Si se captura OTHERS, se informa el error y se vuelve a lanzar con RAISE.

- Ventaja: mensajes más claros. Limitación: hay que definir y mantener cada validación.
Cambios a realizar: Cambiar del tipo varchar2 en id a number